

AI Codebase Mapper

AI-Powered VS Code Development Assistant

User Manual & Guide

Version 0.1.0

Developer: AI Codebase Mapper Contributors

GitHub: [\[GITHUB URL\]](#)

Understand unfamiliar projects faster with AI-generated architecture maps, dependency graphs, and natural-language codebase Q&A.

1. Table of Contents

2. Introduction	3
3. Key Features	4
4. System Requirements	5
5. Installation Guide	6
6. First-Time Setup	7
7. How the Extension Works	8
8. How to Use the Extension	9
9. How to Write Prompts	11
10. Prompt Writing Best Practices	12
11. Prompt Templates	13
12. Example Complete Workflow	14
13. Understanding AI Responses	15
14. Privacy & Security	16
15. Common Problems & Solutions	17
16. Best Practices for Large Projects	18
17. Frequently Asked Questions (FAQ)	19
18. Tips for Getting Better Results	20
19. Support & Contact	21
20. Quick Start Cheat Sheet	22
21. Closing	23

2. Introduction

AI Codebase Mapper is a Visual Studio Code extension that helps developers understand and navigate software projects more easily. It scans the files and folders in your open workspace, analyzes imports and dependencies, and uses Artificial Intelligence to produce a clear architecture map of your codebase.

Large projects can feel overwhelming—especially when you join a new team, inherit legacy code, or switch between many repositories. This extension was created to solve that problem: it turns a maze of folders and files into an interactive overview with modules, entry points, important files, and relationships you can explore and ask questions about.

Who should use it

- Developers joining an unfamiliar codebase
- Engineers reviewing architecture or onboarding teammates
- Students and learners exploring real-world project structure
- Anyone who wants a quick map before diving into the code

AI improves the workflow by summarizing structure, highlighting important files, detecting likely circular dependencies, and answering natural-language questions about how parts of the project fit together—without requiring you to open every file first.

NOTE: This manual describes features available in version 0.1.0. Features marked **Coming Soon** are planned but not yet fully implemented.

3. Key Features

AI-powered project analysis

Scans your workspace, extracts imports/exports, and sends structured metadata (and optional code previews) to an OpenAI-compatible AI model to generate an architecture overview.

File and folder structure visualization

Shows a browsable project file tree. Click a file to open it in the editor.

Interactive architecture graph

Renders modules, entry points, and important files as an interactive graph (zoom, pan, click-to-open). Built with React Flow when the webview UI is available.

Dependency overview

Lists internal import relationships and external packages detected from source files.

Project overview / AI summary

Displays project type, technologies, modules, entry points, potential problems, and suggested improvements returned by the AI.

Natural-language questions about the codebase

Use **Ask AI About Project** to ask questions such as where authentication lives or how a feature is structured. Answers use indexed project context.

File explanations

Use **Explain Current File** (Command Palette or context menu) to get a concise AI explanation of the active file: purpose, symbols, dependencies, and related files.

Large-project navigation

Ignore rules skip node_modules, build outputs, .env, and other noise. File count sent to the AI is capped for performance.

Error handling and guidance

Friendly notifications for missing workspace, invalid API key, network errors, and malformed AI responses. Local fallback map is shown if AI is unavailable.

Status bar access

A status bar item (AI Map) shows analysis progress and opens the architecture map.

Coming Soon

- Full in-webview AI chat panel (chat is currently via Command Palette)
- Persistent cache across VS Code sessions and incremental re-index
- Mermaid diagram export
- Tree-sitter-based deep parsers for more languages

4. System Requirements

Before installing, confirm that your environment meets these requirements.

Requirement	Details
VS Code version	1.85.0 or later (see package engines field)
Operating systems	Windows, macOS, and Linux (VS Code supported platforms)
Internet	Required for AI analysis (calls your configured API endpoint)
AI / API	OpenAI API key or OpenAI-compatible provider (e.g. Azure OpenAI, local gateway)
API key storage	Stored in VS Code Secret Storage — not in settings.json
Permissions	Read access to the opened workspace folder
Optional tools	Node.js 18+ only if building/running from source

INFO: Without an API key you can still scan and view a basic structure map. Full AI architecture analysis, file explain, and Ask AI require a valid key.

5. Installation Guide

5.1 Install from source (development)

Until the extension is published to the Marketplace, install from the project source:

Step 1. Open a terminal and go to the extension folder: `cd ai-codebase-mapper`

Step 2. Run the install script: `bash install.sh` (or install with `npm install`, then `cd webview && npm install && npm run build`, then `npm run compile`).

Step 3. Open the folder in VS Code: `code .`

Step 4. Press **F5** to launch the Extension Development Host.

Step 5. In the new window, open a project folder and use the Command Palette commands under **AI Codebase Mapper**.

5.2 Install from a .vsix file

If you have a packaged .vsix file:

Step 1. Build the package (from the extension folder): `npm run package`

Step 2. In VS Code: Extensions view → ... menu → **Install from VSIX...**

Step 3. Select the generated `ai-codebase-mapper-0.1.0.vsix` file.

Step 4. Or from a terminal: `code --install-extension ai-codebase-mapper-0.1.0.vsix`

Step 5. Reload VS Code if prompted. Confirm the extension appears under Installed extensions.

5.3 Marketplace install (when published)

1. Open VS Code.
2. Open Extensions (**Ctrl+Shift+X** / **Cmd+Shift+X**).
3. Search for **AI Codebase Mapper**.
4. Click **Install**.
5. Verify it appears under Installed. Reload if VS Code asks you to.

NOTE: Marketplace listing may not be available yet. Prefer source or VSIX install for version 0.1.0.

6. First-Time Setup

Complete these steps once after installation.

1. Open a workspace

Action: File → Open Folder... and select your project root.

You should see: The project files in the Explorer sidebar.

If this fails: The extension cannot analyze anything without an open folder.

2. Run Analyze Project

Action: Command Palette → **AI Codebase Mapper: Analyze Project**.

You should see: A progress notification (scanning, dependencies, AI map).

Expected result: Status bar shows “AI Map Ready” and the webview may open.

3. Enter your API key

Action: When prompted, paste an OpenAI or compatible API key.

You should see: Confirmation that the key was saved securely.

Common mistake: Pasting a key with extra spaces or using a revoked key.

4. Recommended settings

Open Settings and search for **AI Codebase Mapper**:

Setting	Recommended default
aiCodebaseMapper.model	gpt-4o-mini (or your preferred model)
aiCodebaseMapper.apiBaseUrl	https://api.openai.com/v1
aiCodebaseMapper.sendSourceCode	true (or false for metadata-only)
aiCodebaseMapper.maxFileSize	100000
aiCodebaseMapper.maxFilesForAI	150
aiCodebaseMapper.autoAnalyze	false

5. If configuration fails

Check the Output panel channel **AI Codebase Mapper** for logs. Use **Clear Project Cache**, verify the API key, and confirm network access to your API base URL. You can still use the local structural map without AI.

7. How the Extension Works

The extension follows a simple pipeline from your workspace to an interactive map.

Workflow

1. You open a project folder in VS Code.
2. You run **Analyze Project** (or open the map after a previous analysis).
3. The extension scans files (skipping `node_modules`, `.git`, `dist`, `.env`, lockfiles, etc.).
4. It extracts imports/exports and builds a dependency graph locally.
5. Structured project metadata (and optional code previews) is sent to your AI provider.
6. The AI returns a validated JSON architecture analysis.
7. The webview shows Architecture, Dependencies, Files, and AI Summary tabs.
8. You click nodes to open files, or use Explain / Ask AI for deeper questions.

Open project → Scan → Dependencies → AI analysis → Interactive map → Ask questions

TIP: Secrets and `.env` files are never scanned. You can disable sending source code via the `sendSourceCode` setting so only paths, imports, and metadata are shared.

8. How to Use the Extension

Launch / open the map

Action: Command Palette → **AI Codebase Mapper: Open Architecture Map**, or click the status bar **AI Map** item.

What you should see: The webview panel titled “AI Codebase Mapper”.

Expected result: If nothing has been analyzed yet, analysis starts automatically.

Common mistake: Forgetting to open a folder first.

Analyze a project

Action: Command Palette → **AI Codebase Mapper: Analyze Project**.

What you should see: Progress notifications: Scanning... Analyzing dependencies... Generating AI map...

Expected result: Status bar: AI Map Ready; webview updates with graph and summary.

Common mistake: Running analyze with no workspace open.

View the architecture graph

Action: Open the **Architecture** tab in the webview.

What you should see: Nodes for project, modules, entry points, important files; edges for relationships.

Expected result: Zoom/pan the graph; click a file node to open it in the editor.

Common mistake: Expecting every file as a node—only key architecture nodes are shown for clarity.

Browse files

Action: Open the **Files** tab.

What you should see: A folder tree of scanned source files.

Expected result: Click a file name to open it in VS Code.

Common mistake: Looking for ignored folders (node_modules, dist)—they are intentionally hidden.

Review dependencies

Action: Open the **Dependencies** tab.

What you should see: Internal edges (source → target) and a list of external packages.

Expected result: Click an internal edge source path to open that file.

Common mistake: Assuming all dynamic imports are resolved—static analysis has limits.

Read the AI summary

Action: Open the **AI Summary** tab.

What you should see: Project type, technologies, modules, entry points, problems, improvements.

Expected result: Use this as a starting orientation, not absolute truth.

Common mistake: Treating every suggestion as a required code change without verification.

Explain the current file

Action: Open a file → Command Palette → **Explain Current File** (or context menu).

What you should see: A side panel with markdown-style explanation.

Expected result: Purpose, symbols, dependencies, risks, related files.

Common mistake: Expecting explanations without an API key configured.

Show dependencies / dependents

Action: With a file open: **Show Dependencies** or **Show Dependents**.

What you should see: A Quick Pick list of related paths.

Expected result: Select an item to open that file (if it is internal).

Common mistake: Running this before any project analysis.

Ask AI about the project

Action: Command Palette → **Ask AI About Project** → type a question.

What you should see: Input box, then a progress notification, then a chat-style panel with the answer.

Expected result: Cited paths and a focused answer based on project context.

Common mistake: Asking questions that need files never included in the scan (e.g. ignored paths).

Refresh or clear cache

Action: **Refresh Analysis** re-scans and re-runs AI. **Clear Project Cache** drops the in-memory snapshot.

What you should see: Progress again on refresh; confirmation on clear.

Expected result: Up-to-date map after major code changes.

Common mistake: Assuming cache persists after restart—v0.1.0 cache is in-memory only.

9. How to Write Prompts

You talk to the AI in normal English. Use **Ask AI About Project** for project-wide questions, and **Explain Current File** when a specific file is already open. Below are example questions you can type into the Ask AI input box.

Understand the project

```
Explain the overall structure of this project in simple terms.
```

Understand a folder

```
What is the purpose of the src folder?
```

Understand a file

```
Explain what src/services/api.ts does and how it is used in the project.
```

Find relationships

```
Which files are responsible for authentication?
```

Understand functionality

```
Where is the login functionality implemented?
```

Find dependencies

```
Explain how App.tsx connects to the pages and services folders.
```

Debugging

```
Help me identify the possible cause of a failed login based on the project structure.
```

Development guidance

```
If I want to add a payment feature, which files would likely need to be modified?
```

Beginner explanation

```
Explain this project as if I am a beginner.
```

10. Prompt Writing Best Practices

- **Be specific.** Name folders, files, or features when you can.
- **Mention paths.** Example: “src/hooks/useAuth.ts” is better than “the auth hook”.
- **Describe the output you want.** Overview, step-by-step, list of files, risks, etc.
- **Ask one main question at a time.** Follow up after you get an answer.
- **Paste error messages** when debugging so the AI can relate them to structure.
- **Request simpler language** if the first answer is too dense.

Examples: bad → better → best

Bad:

```
Explain this.
```

Better:

```
Explain the purpose of the src/components folder and describe how its files are related.
```

Best:

```
Analyze the src/components folder and explain the responsibility of each major component, how the components interact, and which files I should check if the UI is not rendering correctly.
```

11. Prompt Templates

Copy these templates and replace the bracketed parts.

Analyze [FILE/FOLDER] and explain its purpose.

Explain how [FEATURE] works in this project.

Find the files related to [FEATURE].

Explain the relationship between [FILE A] and [FILE B].

I want to add [FEATURE]. Which files should I modify and why?

Help me debug [ERROR]. Identify the likely files involved and explain why.

Give me a beginner-friendly overview of this project.

List the entry points of this project and what each one starts.

Are there circular dependencies I should worry about?

Summarize the technologies used and where each one appears.

12. Example Complete Workflow

Imagine you open a React + TypeScript app with folders like `src/pages`, `src/components`, and `src/services`.

1. Open the project folder in VS Code.
2. Run **AI Codebase Mapper: Analyze Project** and enter your API key if prompted.
3. Wait until the status bar shows **AI Map Ready**.
4. Open the Architecture tab: you see modules such as Frontend, entry points like `src/main.tsx`, and important files like `App.tsx`.
5. Ask AI: *Explain the architecture of this project.*
6. Read the answer: technologies (React, TypeScript), modules, and data flow notes.
7. Follow up: *Where is authentication implemented?*
8. Open the cited files (e.g. `useAuth.ts`, `Login.tsx`) from the Files tab or by clicking graph nodes.
9. Continue development with a clearer mental model of the codebase.

13. Understanding AI Responses

AI output is a helpful orientation layer on top of real scan data. Treat it as a guide, not as a substitute for reading the code.

- **Interpret structure claims** by opening the cited paths in the editor.
- **Verify suggestions** before changing production code or architecture.
- **When the AI says “unknown”**, the scan context was insufficient—narrow the question or open the file and use Explain.
- **Follow-up questions** work best when you refer to paths from the previous answer.
- **Request detail or simplicity** explicitly (“in three bullets”, “for a beginner”).

WARNING: AI-generated suggestions may not always be completely accurate. Always verify important code changes before applying them.

14. Privacy & Security

Understanding what leaves your machine is essential.

Topic	Behavior in AI Codebase Mapper v0.1.0
What is scanned	Source and config files under the workspace, subject to ignore rules
Never scanned	.env, .env.*, secret-like names, lockfiles, node_modules, build outputs, .git
Sent to AI	When sendSourceCode is true: metadata + truncated previews. When false: metadata and dependencies only
API key	Stored in VS Code Secret Storage; never hard-coded; not written to settings.json
Network	Outbound calls only to your configured apiBaseUrl for AI features
Redaction	Common secret patterns are redacted before Explain/Chat content is sent

Recommended practices

- Use a restricted API key with spending limits.
- Disable sendSourceCode for highly sensitive repositories.
- Do not put secrets in filenames that look like normal source files.
- Review ignore patterns and add company-specific paths via excludePatterns.

15. Common Problems & Solutions

Problem	Possible cause	Solution
Extension commands missing	Not installed / host not loaded	Install or press F5 in Extension Development Host; reload window
AI does not respond	Missing or invalid API key	Re-run Analyze and enter a valid key; check apiUrl
Project analysis fails	No folder open	File → Open Folder... then Analyze Project
Empty or weak map	Most files ignored or non-source	Confirm you opened the real project root; check excludePatterns
Incorrect AI info	Limited context / model error	Refresh analysis; ask a more specific prompt; open files to verify
Extension feels slow	Very large repo	Lower maxFilesForAI; set sendSourceCode false; add exclude patterns
API error 401	Bad key	Clear and re-enter key; verify provider dashboard
Timeout	Network or large payload	Retry; reduce maxFilesForAI and maxFileSize
File will not open from graph	Path outside workspace or external package	Only internal project files can open; externals are packages
Cache seems stale	In-memory cache	Run Refresh Analysis or Clear Project Cache

16. Best Practices for Large Projects

- Open the correct repository root (the folder that contains package.json or the main source tree).
- Rely on default ignore rules; add extra globs under excludePatterns for monorepo noise.
- Keep maxFilesForAI at a reasonable value (default 150) so AI context stays focused.
- Ask targeted questions about one module or feature instead of “explain everything”.
- Re-run **Refresh Analysis** after large refactors so the map matches the code.
- Use Show Dependencies / Dependents on critical files to navigate without the full graph.
- For private monorepos, consider sendSourceCode: false and rely on structure + imports.

17. Frequently Asked Questions (FAQ)

Q: What does this extension do?

A: It scans your VS Code workspace, builds a dependency picture, and uses AI to produce an architecture map and answers about the codebase.

Q: Who can use it?

A: Any developer using VS Code 1.85+ who can open a project folder. AI features need an API key.

Q: Does it require an API key?

A: Yes for AI architecture analysis, Explain, and Ask AI. Scanning and a basic local map can work without one.

Q: Can it analyze large projects?

A: Yes, with ignore rules and caps on file size and number of files sent to the AI. Extremely large monorepos may need extra exclude patterns.

Q: Can I ask questions in natural language?

A: Yes. Use Ask AI About Project and type questions in ordinary English.

Q: Can I ask about a specific file?

A: Yes. Open the file and run Explain Current File, or mention the path in an Ask AI question.

Q: Can I ask about a specific folder?

A: Yes. Include the folder path in your prompt (for example, "What is the purpose of src/services?").

Q: Can I use it for debugging?

A: It can suggest likely files and structure-related causes. It does not replace a debugger or test suite.

Q: Can I refresh the project analysis?

A: Yes. Run Refresh Analysis or Clear Project Cache then Analyze again.

Q: Is my source code sent to an AI provider?

A: Only if sendSourceCode is true (default). You can turn it off to send metadata and dependencies only. .env and secret-like files are never sent.

Q: Which languages are supported for import analysis?

A: TypeScript/JavaScript, Python, Java, Go, Rust, and C# have dedicated extractors; other files are listed with limited analysis.

Q: Does the cache survive VS Code restart?

A: Not in v0.1.0—the cache is in-memory. Persistent cache is planned (Coming Soon).

18. Tips for Getting Better Results

- Ask specific questions and name files or folders.
- Give short context (“we use JWT auth”, “this is a Next.js app”) when it is not obvious from the tree.
- Ask follow-up questions that build on the previous answer.
- Request structured answers (“list five files”, “use bullet points”).
- Request step-by-step explanations for onboarding.
- After AI points to a file, open it and use Explain Current File for depth.
- Keep the webview open while you explore so you can jump between graph and editor.
- Check the Output channel “AI Codebase Mapper” if something fails silently.

19. Support & Contact

Resource	Value
Extension name	AI Codebase Mapper
Version	0.1.0
GitHub repository	[GITHUB URL]
Documentation	This User Manual + project README.md
Issue tracker	[GITHUB URL]/issues
Developer / contact	[SUPPORT EMAIL]
Website	[WEBSITE URL]
VS Code engines	^1.85.0

When reporting issues, include VS Code version, OS, whether the API key is configured, and a brief description of the workspace (language, approximate size). Do not paste secrets.

20. Quick Start Cheat Sheet

Keep this page handy for daily use.

Step 1: Open your project folder in VS Code.

Step 2: Command Palette → AI Codebase Mapper: Analyze Project.

Step 3: Enter API key if prompted; wait for “AI Map Ready”.

Step 4: Open Architecture Map (command or status bar).

Step 5: Explore Architecture, Dependencies, Files, and AI Summary tabs.

Step 6: Click a node or file to open it in the editor.

Step 7: Ask AI About Project — type a natural-language question.

Step 8: Explain Current File for deep dives; Refresh after major changes.

Essential commands

Command	Purpose
Analyze Project	Full scan + AI map
Open Architecture Map	Show webview
Refresh Analysis	Re-scan and re-analyze
Explain Current File	AI explanation of active file
Show Dependencies	Outgoing imports
Show Dependents	Who imports this file
Ask AI About Project	Natural-language Q&A
Clear Project Cache	Drop in-memory snapshot

21. Closing

*Thank you for using **AI Codebase Mapper**. We hope it helps you understand your codebase faster, navigate complex projects more easily, and build better software with the power of AI.*

AI Codebase Mapper v0.1.0 — User Manual
For updates, issues, and contributions, see [\[GITHUB URL\]](#).